CSC4010: Parallel and Distributed Computing

Assignment 1: Parallel Computing



Version and Date	1 (2026-10-08)
Date Set	14th October 2026
Due	**2nd November 2026 17:00**
Title	CSC4010 Assignment 1: Parallel Computing
Weighting	50% of module
Set By	David Cutting
Moderated By	Ciaron Fitzpatrick

Late submission penalties and rules will be applied in accordance with the QUB policy on late submission. For more information on this or any other QUB policy with regard to assessment please see the academic affairs guidance.

Please be aware that this is an individual assignment and as such will be checked for plagiarism. You should ensure that the work is yours and yours alone, citing any third-party sources as applicable. Plagiarism is a serious academic offence. Submission of your work implies your claim that it is your individual work and has not previously been submitted for academic credit.

It is the responsibility of the student to ensure the correct items are submitted (they can be downloaded and checked after submission). Where multiple submissions are allowed the final submission closest to the deadline will be marked.

If you have any questions about the assignment please see the module organiser David Cutting in the first instance or use any of the support options listed in this document.

1 Overview

The purpose of this assessment is to demonstrate your knowledge, understanding and application of the principles and tools of parallel computing. We will do this using a simple pattern matching algorithm in a number of scenarios which are provided pre-written in sequential form along with sample source and pattern data files.

You will be required to plan, implement and measure parallelisation of the provided sequential code.

Frequency analysis, the identification of commonalities and regular occurrences in plain or encrypted text can be a key part of cryptographic analysis and code breaking. You have been asked by the *QUB Secret Squirrel Investigations Division* to help perform some frequency analysis against plain text samples they believe are being used by enemy agents.

You will do this utilising high-performance computing to search provided texts for given patterns, listing their occurrences for further analysis and also their frequencies. We have existing sequential code to perform this work but it is very slow and we now need to use HPC resources and OpenMP to speed it up.

The provided sequential code operates in one of two methods (A and B) where a source text file is loaded along with a pattern file and the source is searched for the pattern(s).

Method A: The source data is broken into chunks/lines of a fixed length (1024 bytes by default) forming a data structure of `char[][]` with the first index being the chunk/line number and the second being the character of that chunk/line.

For each pattern, each source chunk is searched and the index of the start of the first occurrence of this pattern is returned from the search (or -1 if the pattern is not found).

The method is required to output, to standard out, a line (with a newline at the end) for each match found in the format:

```
**,pattern,chunk_num,index
```

For example if the pattern `letter` is found in chunk `74` starting at index `19` the output should be:

```
**,letter,74,19
```

Method B: The source data is loaded as a single chunk of full length (using the same data structures as A so all the data is contained in chunk `[0]`).

For each pattern the source data is searched and the number of instances of the pattern found returned (0 if the pattern was not found).

The method is required to output, to standard out, a line (with a newline at the end) for each pattern in the format:

```
**,pattern,count
```

For example if the pattern `letter` is found twice in the source `I sent a letter and got a letter back` the output would be:

```
letter,2
```

2 Provided Code and Data

You will be provided, via Canvas, a zip file containing `search.h` the implementation of a basic file loading and naive search algorithm (see the notes on changing this file) along with `method-a.c` and `method-b.c` which are sequential implementations of methods A and B. There will also be a `searchtest.c` file showing the searching working on a small static set of data.

Editing `search.h`: Although you are welcome to edit this file as you need to debug your programs for the assessed execution your code will be compiled with the original version (this ensures everyone has the same naive [aka inefficient!] search routine and file loading overhead). Bear this in mind when making any changes.

There are a number of example source and pattern data available of different sizes which you can download directly to your machine (and/or download directly to Kelvin using the wget command) which can be used to

test and assess your implementations. Source text is randomly generated from words within Project Gutenberg free copies of Pride and Prejudice, Fabian Essays in Socialism, and Moby Dick with words separated with an underscore (_).

2.1 Source Data

- `https://files.altair.davecutting.uk/CSC4010/data.zip` contains 2 x 1MB and 2 x 100MB source data files.
- `https://files.altair.davecutting.uk/CSC4010/data-large.zip` contains 1 x 1GB data file.

2.2 Pattern Data

- `https://files.altair.davecutting.uk/CSC4010/patterns.zip` contains 2 small pattern files.

3 Your Task

Having been provided with a sequential version of methods A and B you must plan, implement, and evaluate parallel solutions for both using OpenMP.

You are expected to define approaches to parallelisation, implement them and then experiment with your code (i.e. different settings, structure and configuration) to determine what impact on overall execution time (including through measurements of Parallel Speedup and Parallel Efficiency) they have as well as identifying any other relevant factors, before producing code and a detailed report of your findings.

You are expected to submit a report along with your fastest (highest PE) versions of code for method A and B (see 6 for detailed instructions). Marking will be based on (a) the quality and completeness of the report showing implementation, experimentation, findings and other factors and (b) the timing of your code run on 16 threads against unseen source and pattern files (the mean of the best 8/10 wall times of execution will be used as your recorded time).

Some hints which may (or may not!) be useful depending on how you implement your solutions:

- Memory use is not a factor for execution (you may wish to comment on it as a trade-off for example in your report but there is no requirement for your code to be memory efficient).
- The timed execution will take place for a large (1GB+) source file and a pattern file containing 10 or fewer patterns.
- When tested for correctness of output only lines beginning `**,` will be used i.e. you can put as much debug or other output in as you like (though of course that may slow things down).

4 Assessment Criteria

Each method will be marked out of 50 (making an assessment total out of 100) and marks will be awarded against the following criteria.

Quality of Implementation and Investigations (15 marks) - judged from the report this will include the approach used to parallelisation and discussion/consideration along with relevant experiments showing how different setups affect (or not) performance.

Quality of Report (15 marks) - how well the report is presented including the quality of figures and explanation of work (the first criterion is the technical quality of proposed solutions, that is how well they are explained and justified in the report).

Speed of Execution (10 marks) - how quickly the code executes a standard (unseen) set of source and pattern data roughly ranked against class submissions. Note if your output is incorrect then between 5 and 10 marks will be lost here (5 for minor issues perhaps with formatting, 10 for incorrect results).

Wow Factor (10 marks) - anything above and beyond what would *normally be expected* (i.e. trying things that have been covered purely in the taught material) that impresses in *any* (so wow points available for great quality of reporting as well as great coding) part of the assignment.

5 Feedback

Individual feedback including marks and textual commentary will be provided as soon as possible after submission, targeted within two weeks. Once marks and feedback are received anyone wishing to discuss this in more detail is welcome to do so with the module owner.

Some generalised class-wide feedback may also be provided.

6 Submission

Submission is via Canvas by the due date and time.

You should submit **a single zip file** containing your fastest (highest PE for 16 processors) implementation of a parallel method A and B **called method-a-p.c and method-b-p.c** along with a **PDF file report** all in a folder of your student number. So your zip should just contain a top level folder of your student number in turn containing three files (note for ease you are welcome to submit your search.h file but this will be swapped for the original version during evaluation).

Code examples (such as other versions of A and B you are not submitted for final evaluation) should not be submitted in the zip but can be included in the PDF report.

Please make sure your zip file follows the format above as failing to do that will make my life even more miserable in extracting and running them, and I shall **curse your code**.

7 Use of AI

AI is *just another tool* (or maybe the ultimate tool, who knows) and you are welcome to use it to help you develop ideas however you must (1) acknowledge the use of AI as appropriate in your report and (2) ensure you understand your work and that it is your own.

8 Verbal Discussion / Viva

Any student submitting to this assignment may be asked to have a discussion with the module lead (a mini-viva) about their submission as part of the marking process.

9 Support Available

CSC4010 offers office hours every week bookable via Canvas. You are also welcome to ask questions in or after any of the teaching activities or contact the module lecturer directly (teams is best and email also works).